

Backlinks (1) ▾

Porehammer2 code

1. [About the Code](#)
 - b. [Required Libraries](#)
 - c. [Overall Inputs and Outputs](#)
 - d. [File Structure](#)
 - e. [Naming Convention](#)
6. [Code use](#)
 - g. [Squaremaker](#)
 - h. [Pathfind](#)
9. [Some Notes](#)

About the Code

Porehammer2 is the updated set of Python code used to generate the porous media. It follows the process outlined in Abitbol et al., which

- Generates a randomized lattice of links of random radius ranging from $0.17x$ to $0.32x$ link length
- closes 1-p links from largest to smallest radii
- assigns a link weight τ , which represents the amount of shear stress needed to progress past the link
- uses Dijkstra's algorithm to find the "chemical path" and calculate the total shear stress needed to progress this path, which is equated to ΔP of the system
- uses Newman-Ziff to calculate the percolation threshold p_c of the medium

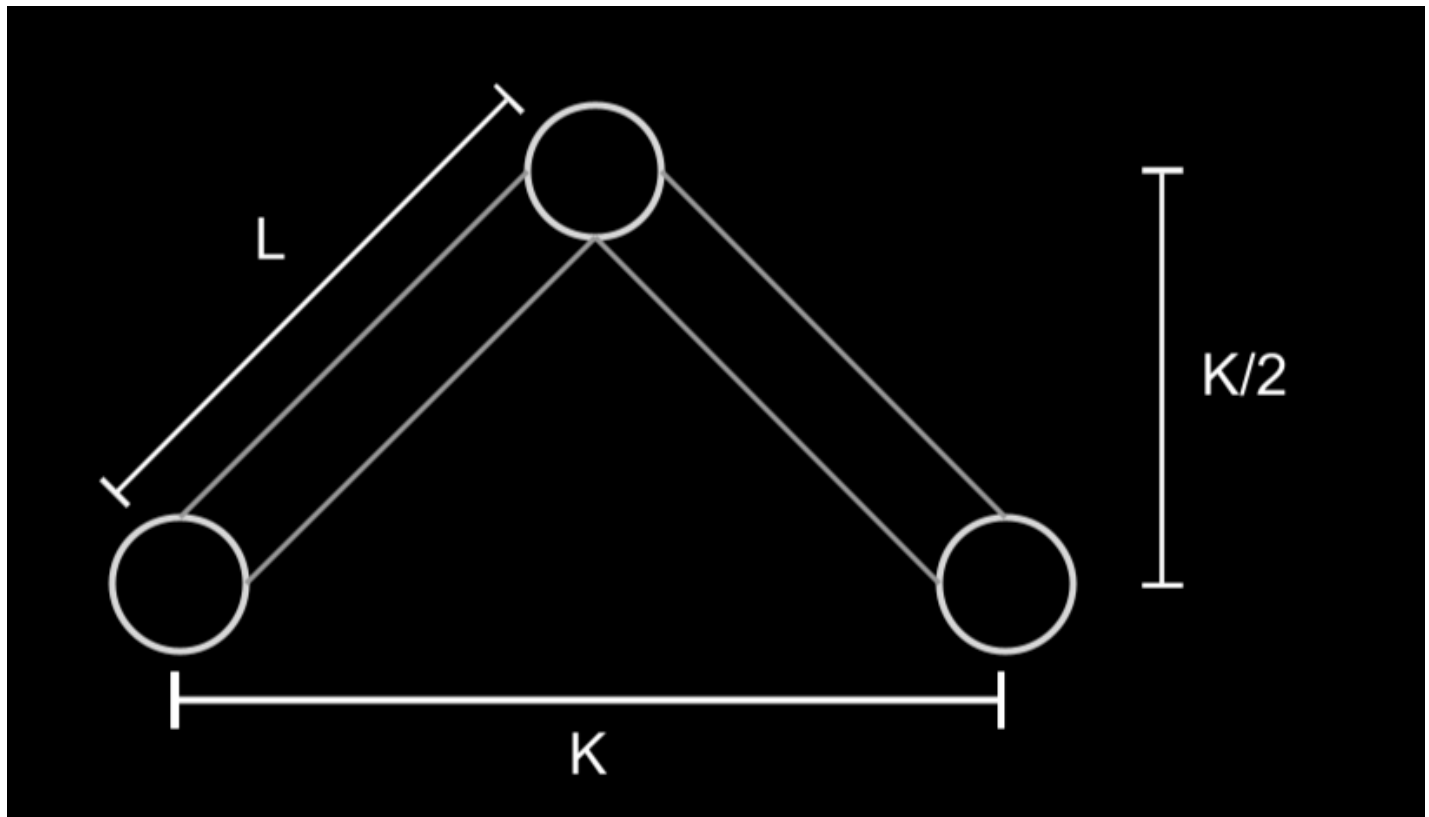
Required Libraries

- matplotlib
- networkx
- numpy
- cadquery

Overall Inputs and Outputs

The code has inputs sample size L , scale factor k , sample thickness h , desired percolation fraction p ,

and random number seed SD. The random number seed allows the randomness in the sample to be controlled and repeatable; ie, a sample of the same size and random number seed will generate the same network geometry every time, and the percolation fraction controls how many of those links are closed. The scale factor K is defined as the desired lateral distance between 2 nodes, in mm.



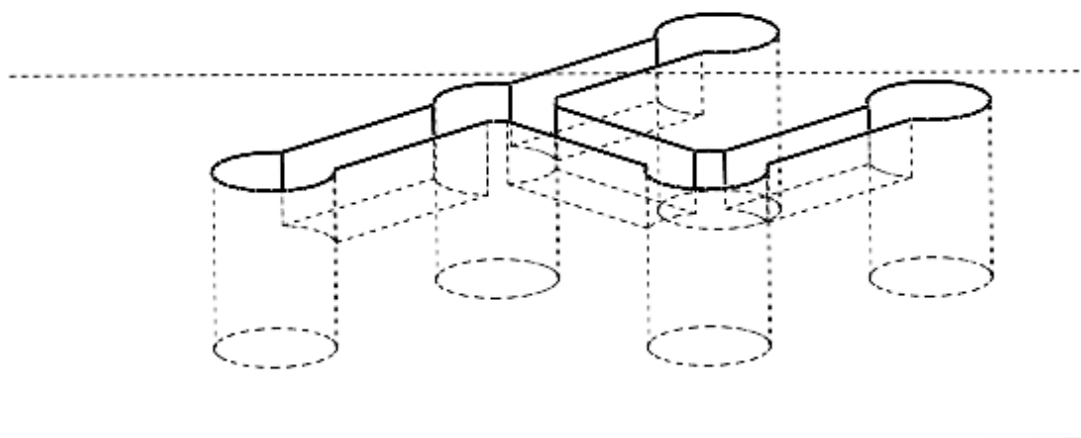
The code outputs a network diagram, a network diagram with the predicted path, and a STEP file. It will also indicate the percolation threshold and predicted system ΔP in the console. Both diagrams are exported as a PNG file.

While I have optimized the code to the best of my abilities, generating a porous medium larger than $L=128$ is not advised. Generating a medium of size $L=128$ took 90 minutes and produced a 178 MB STEP file. It should be noted that, once the STEP file is imported and worked on in SolidWorks, a smaller SLDPRT file would be generated. That being said, proceed at your own risk.

I also suggest breaking the link between the SLDPRT and the STEP file as it reduces the likelihood of the file breaking. If the screen is blank upon editing a previously worked on SLDPRT with a porous medium, check if the part itself is hidden in SolidWorks or try rebuilding the file.

All diagrams have the leftmost column of nodes as the inlet and the rightmost column of nodes as the outlet.

The exported STEP file will be a square block with the network situated in the first quadrant, which means some trimming in CAD will be needed. The links will be rectangular cutouts with a square profile. The geometry of these links have been calculated to behave the same as if they were circular. The links will share a plane on which their base sit, as shown below.



File Structure

The file structure of Porehammer2 is as follows:

```
porehammer2/
├── main.py
├── loop.py
├── dijkstra.py
├── newmanziff.py
├── poremaker.py
├── squaremaker.py
├── outputs/
│   ├── figures/
│   └── step/
```

- "outputs", "figures", and "step" must be in the same folder as all of the Python files and the file structure must be the same as outlined above
- loop.py allows the user to generate multiple samples at the same time, which I'm sure you will be able to modify to better fit your needs
- poremaker.py is retired in Porehammer2 as it generates circular links centred in the middle of the medium, which did not match manufacturing limitations. That being said, the module is updated

alongside squremaker.py and remains useable.

Naming Convention

The STEP files generated by Porehammer2 follow the naming convention

Auto (HTML, XML) ▾



`SD<seed>-p<percolation_fraction>-<size>n-<scale>x<height>.step`

Example:

`SD42-p0.8-64n-3x5.step`

`SD42-pc-64n-3x5.step`

The PNG files generated by Porehammer2 follow the naming convention

Auto (HTML, XML) ▾



`SD<seed>-p<percolation_fraction>-<size>n.png`

Example:

`SD42-p0.8-64n.png`

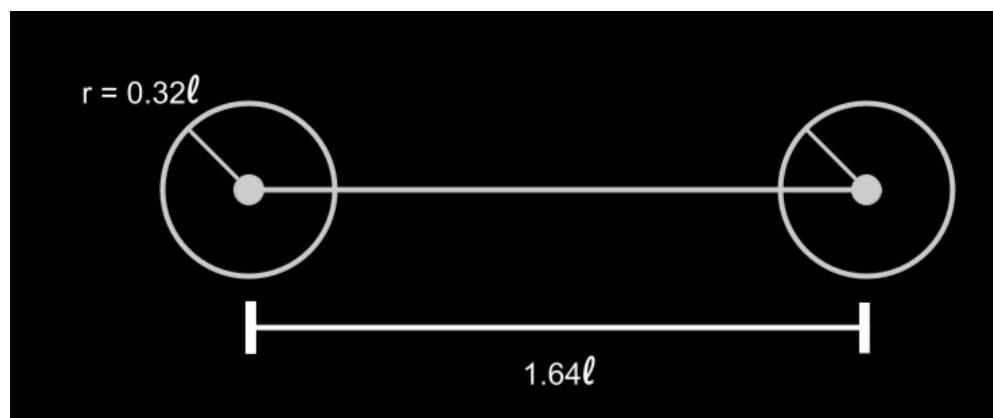
`SD42-pc-64n.png`

`SD42-p0.8-64n-path.png`

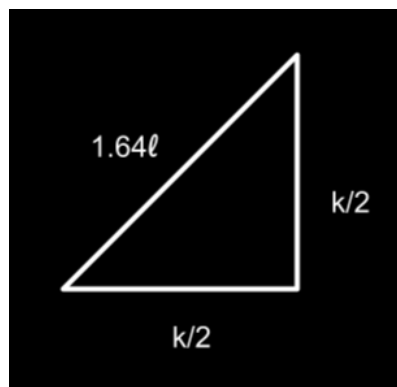
`SD42-pc-64n-path.png`

You will notice that the definition of link length is a percent of the scale factor. The underlying math is as follows.

- Define node radius as the maximum link radius, rounded to the next tenth of a mm, let r_{node} be 0.32ℓ , which is the upper bound for link radius, where ℓ is the link length.



- Recall the definitions for ℓ and k , and note that



- Which means we can then say

$$1.64^2 k^2 = \frac{k^2}{2}, \quad l = \frac{k}{1.64\sqrt{2}}$$

Given a sample of size L at a scale of k , the centre of the rightmost node to the centre of the leftmost node follows

$$\frac{1}{2}k(L-1)$$

As the sample is a square, this dimension also applies to the distance from the centre of the top node to the centre of the bottom node.

Code use

This section will be seperated by modules.

Squaremaker

poremaker.py behaves similar to squaremaker.py and has been continuously updated alongside it, but poremaker is no longer in use due to physical manufacturing constraints. Every function in Squaremaker is in Poremaker with the exception of equivalent_square(). There is also an additional function in Poremaker called cry().

`makemesh(size, scale, p, seed=42, showGraph=False, exportGraph=True, theta=0)`

- **size:** desired network size L; int
- **scale:** scale factor k; float
- **p:** the percentage of links to be left open; should be the string 'pc' if the percolation threshold is desired; float (0 ~ 1), string of value 'pc'
- **seed:** random seed for reproducible randomness; integer, float, str, bytes or bytearray;
- **showGraph:** indicates if the network graph is to be shown during runtime, which interrupts the script until the graph is closed; bool
- **exportGraph:** indicates if the network graph is to be exported; bool
- **theta:** the contact angle of the defending fluid with the channel walls in radians; int, float
- returns G, positions, and percolation threshold
 - **G:** the Networkx graph containing the percolation network
 - **positions:** dictionary containing the physical coordinates of each node in mm; dictionary
 - **percolation threshold:** the percentage of open links where the first flow path is established; float
- if exportGraph is on, it will export a png file in /outputs/figures

Example

Python ▾



```
import squaremaker as sm
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
```

buildmesh(mesh, p, pos, height, size, scale, seed=42)

- **mesh:** the Networkx graph returned by makemesh() that stores the network and link attributes
- **p:** the percolation fraction of the mesh; unlike makemesh(), **this must be a float (0 ~ 1)**
- **positions:** the dictionary containing the node coordinates returned by makemesh()
- **height:** the thickness of the desired STEP file; int, float
 - **there are no safeguards checking if the height is greater than the largest resulting link**
- **size:** desired size L of the network
- **scale:** scale at which the network is to be made
- **seed:** the random seed used to generate the network

- exports a STEP file in /outputs/step
- computationally expensive due to geometry calculations

Example

Python ▾



```
import squaremaker as sm
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
sm.buildmesh(G, pc, 5, L, k, seed=36)
```

equivalent_square(r)

- **r**: radius of cylindrical channel/pore of uniform profile
- returns the width of a square channel such that it has the same Poiseuille conductance as a cylindrical channel of radius **r**.

Example

Python ▾



```
import squaremaker as sm
a = sm.equivalent_square(r)
```

make_uniform(size, scale, p, seed=42, showGraph=False, exportGraph=False, theta=0)

- **size**: desired network size **L**; int
- **scale**: scale factor **k**; float
- **p**: the percentage of links to be left open; should be the string 'pc' if the percolation threshold is desired; float (0 ~ 1), string of value 'pc'
- **seed**: random seed for reproducible randomness; integer, float, str, bytes or bytearray;
- **showGraph**: indicates if the network graph is to be shown during runtime, which interrupts the script until the graph is closed; bool
- **exportGraph**: indicates if the network graph is to be exported; bool
- **theta**: the contact angle of the defending fluid with the channel walls in radians; int, float

- returns **G**, positions, and percolation threshold
 - **G**: the Networkx graph containing the network where every open link has the same size
 - **positions**: dictionary containing the physical coordinates of each node in mm
 - **percolation threshold**: the percentage of open links where the first flow path is established, float
- if exportGraph is on, it will export a png file in /outputs/figures

Example

Python ▾



```
import squaremaker as sm
L = 32
k = 4
G, pos, pc = sm.make_uniform(L, k, 'pc', exportGraph=True, seed=36, theta=0)
```

Pathfind

active_net(mesh: networkx.Graph)

- **mesh**: Networkx graph generated using Squaremaker or Poremaker; Networkx graph
- returns **active**: Networkx graph that only has the active links, which saves computation time for generation and pathfinding.

Example

Auto (Python) ▾



```
import squaremaker as sm, pathfind as pf
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
active = pf.active_net(G)
sm.buildmesh(active, 'pc', pos, h, L, k, seed=36)
```

dijkstra(mesh: networkx.Graph, param: str)

This function is used for the single-phase percolating network case, not the two-phase fully

opened case. This function is not currently being used in main.py for this reason.

- **mesh**: the Networkx graph generated using Squaremaker or Poremaker; Networkx graph
- **param**: the parameter that the algorithm uses to search for the optimal path and its corresponding cost, which must be a key in found in the link parameters of the Networkx graph; string
- returns **path** and **cost**
 - **path**: a list of nodes in the shortest path; list
 - **cost**: the cost corresponding to the sum of the selected parameter along the optimal path; float

Example

Python ▾



```
import squaremaker as sm, pathfind as pf
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
active = pf.active_net(G)
sm.buildmesh(active, 'pc', pos, h, L, k, seed=36)
path, cost = pf.dijkstra(active, 'tau')
```

show_min_path(mesh: networkx.Graph, pos, path)

- **mesh**: the Networkx graph generated using Squaremaker or Poremaker; Networkx graph
- **pos**: dictionary containing the physical coordinates of each node in mm; dictionary
- **path**: list of nodes in the shortest path; list
- shows a pyplot window that highlights the shortest path
- will interrupt the script until the window is closed

Example

Auto (Python) ▾



```
import squaremaker as sm, pathfind as pf
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
```

```
active = pf.active_net(G)
sm.buildmesh(active, 'pc', pos, h, L, k, seed=36)
path, cost = pf.dijkstra(active, 'tau')
pf.show_min_path(G, pos, path)
```

save_min_path(mesh: nx.Graph, pos, path, p, pc, seed=42)

- **mesh**: the Networkx graph generated using Squremaker or Poremaker; Networkx graph
 - **pos**: dictionary containing the physical coordinates of each node in mm; dictionary
 - **path**: list of nodes in the shortest path; list
 - **p**: degree of percolation of the network, ie, how open the network is; float
 - **pc**: the percentage of open links where the first flow path is established; float
- exports a graph of the network that highlights the optimal path to /outputes/figures

Example

Auto (Python) ▾



```
import squaremaker as sm, pathfind as pf
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
active = pf.active_net(G)
sm.buildmesh(active, 'pc', pos, h, L, k, seed=36)
path, cost = pf.dijkstra(active, 'tau')
pf.save_min_path(G, pos, path, pc, pc, seed=36)
```

twoP_pressure(mesh: networkx.Graph)

- **mesh**: the Networkx graph generated using Squremaker or Poremaker; Networkx graph
 - this requires the network to have link weights of 'tau' and 'P_cap'
- returns **capillary** and **yield stress** contributions
 - **capillary contribution**: the amount that capillary pressure contributes to the total breakthrough pressure
 - **yield stress contribution**: the amount that yield stress contributes to the total breakthrough

pressure

Example

Auto (Python) ▾



```
import squaremaker as sm, pathfind as pf
L = 32
k = 4
G, pos, pc = sm.makemesh(L, k, 'pc', exportGraph=True, seed=36, theta=0)
cap, shear = pf.twoP_pressure(G)
cost = cap + shear
```

Some Notes

- squaremaker and poremaker generate different geometries
 - squaremaker generates a network of square links extruded from a common face of the sample
 - poremaker generates a network of cylindrical links whose axes lie in the centre of the sample
 - poremaker generates a network closer to Abitbol et al. but is less manufacture-friendly (more prone to defects)
 - squares from squaremaker have geometries of equal Poiseuille conductance to the cylindrical links, based on Boussinesq's work as presented in Frank M. White (§3-3.3)
 - makemesh() is identical between squaremaker and poremaker except in the definition of capillary pressure
 - hydraulic conductivity is the same between squaremaker and poremaker as equivalent hydraulic conductivity was used to determine the size of the square.
- Update t_c and tension in both squaremaker and poremaker as needed to reflect the values of the fluid being used.
- makemesh() must be run before buildmesh()
 - makemesh() generates the network Graph object, percolation threshold, and node positions, and declares global variables used in buildmesh()
 - makemesh currently only generates an $L \times L$ network with every second column offset by half of node spacing
- buildmesh() is not very well optimized and take quite a while to generate larger networks. Currently, the largest network that has been generated is $L \sim 128$
 - 128x128 makes a large STEP file, proceed at your own risk.

